

The screenshot shows the Spring Initializr configuration page. It is divided into several sections:

- Project:**
 - ☒ Maven Project
 - ☐ Gradle Project
- Language:**
 - ☒ Java
 - ☐ Kotlin
 - ☐ Groovy
- Spring Boot:**
 - ☐ 2.5.0 (SNAPSHOT)
 - ☐ 2.5.0 (M3)
 - ☐ 2.4.5 (SNAPSHOT)
 - ☒ 2.4.4
 - ☐ 2.3.10 (SNAPSHOT)
 - ☐ 2.3.9
- Project Metadata:**
 - Group:** com.example
 - Artifact:** demo
 - Name:** demo
 - Description:** Demo project for Spring Boot
 - Package name:** com.example.demo
 - Packaging:** ☒ Jar ☐ War
 - Java:** ☐ 16 ☒ 11 ☐ 8
- Dependencies:**
 - Lombok** (DEVELOPER TOOLS): Java annotation library which helps to reduce boilerplate code.
 - Spring Session** (WEB): Provides an API and implementations for managing user session information.

At the bottom, there are three buttons: **GENERATE** (CTRL + G), **EXPLORE** (CTRL + SPACE), and **SHARE...**

Figure 1: Spring Initializr

Initialising a Spring Boot project

It's very easy to get started with a Spring Boot application. For developers, there is a Web application called Spring Initializr that builds a bare bones Spring Boot application, which is preconfigured in its opinionated design architecture. It can be found at <https://start.spring.io/> and has modern IDE connectors such as IntelliJ IDEA.

Spring Initializr lets you select the Java project type (Maven or Gradle), language (Java, Kotlin, or Groovy) and project details. After that, the project's dependencies can be chosen.

A new Spring Boot project is created when you complete the steps. The project structure for Spring Boot projects is explained below.

Details of layers

Spring divides a project into layers for all developers to follow, allowing for a convention-over-configuration approach. This allows for a clear understanding of each component and the separation of concerns by isolating their functionality. Spring does this by managing the objects that will be utilised in a Spring application and allowing them to communicate with one another in a seamless and methodical manner. Spring Beans and dependency injection are used to accomplish this.

A Spring Bean is an object which, along with other beans, forms the application's backbone. It is created automatically using configuration metadata in the Spring context, and for its lifetime, it is contained in the Spring

IoC container. Spring managed beans include controllers, services, repositories and components, which can be injected as dependencies for other Spring beans with the `@Autowired` annotation, either on field variables or on top of constructors. This enables inversion of control and separation of concerns.

Field injections are not generally recommended for use but preferably injected as dependencies in the constructors. This makes it easier to write unit tests without loading the Spring application context. Let's understand the layers used in a typical Spring Boot application in depth.

Controllers: As Spring Boot projects are generally Web projects, either working as backends for REST services or server-side rendered UI, the points of contact between the client (browsers/REST consumers) are the controllers. Controllers expose endpoints for API consumers to use. Spring provides automatic serialisation and deserialisation of request and response objects from JSON to POJOs, which can be essentially treated as Data Transfer Objects. Spring comes bundled with Jackson — a powerful JSON to POJO serialisation library. Controllers are the entry and exit points for the outside world to interact with our application. They are generally logicless and only deal with data transfer to service layers with some transformation.

The `@Controller` or `@RestController` annotations define a controller. `@Controller` returns a server rendered view with data in the form of request body or request parameters, whereas `@RestController` produces a `ResponseBody` object, which is essentially domain or data transfer objects converted to a content type like